UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (12).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Component X is the integrated circuit (IC), also known as a microchip/chip — a complete electronic semiconductor circuit fabricated on a piece of silicon.

(b) The integrated circuit was introduced in the third computer generation (1964–1971), the generation characterised by ICs replacing discrete transistors.

(c) The integrated circuit was invented by Jack Kilby (Texas Instruments, 1958), with Robert Noyce (Fairchild, 1959) independently developing a practical silicon IC; Jack Kilby is the conventional textbook answer.

(d) Input device: keyboard (also acceptable: mouse). Output device: monitor (also acceptable: printer). These were introduced/used in the third generation alongside ICs.

Unofficial marking guide:

- *1 mark: (a) integrated circuit / IC / chip*
- *1 mark: (b) third generation*
- *1 mark: (c) Jack Kilby (accept Robert Noyce)*
- *1 mark: (d) one correct input device (keyboard or mouse)*
- *1 mark: (d) one correct output device (monitor or printer)*

Model confidence: high.

Model note: Figure 1 image not inspected; answer relies on the extracted text describing a silicon chip. Some syllabi credit Noyce equally for the IC; both accepted.

QUESTION A2 (5 marks)

Topic: Data representation and number systems

Answer: left -31 -> 1 0011111; right -12 -> 1 0001100; -31 - (-12) = -19 -> sign-magnitude 1 0010011

- *1 mark for converting both operands to sign-magnitude.*
- *1 mark for applying the sign rule for subtraction.*
- *1 mark for the correct magnitude operation.*
- *1 mark for the correct sign bit.*
- *1 mark for the final decimal and byte representation.*

QUESTION A3 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (8).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Classification of parallel processors is constructed according to the relationship between the instruction stream and the data stream, as proposed in Flynn's taxonomy. A parallel processor is described by whether it issues one or many instruction streams and whether it operates on one or many data streams, giving four categories: SISD (single instruction, single data), SIMD (single instruction, multiple data), MISD (multiple instruction, single data) and MIMD (multiple instruction, multiple data). This construction classifies architectures by their parallelism at the instruction and data level.

(b) One example for each parallel processor architecture: SISD — a conventional single-core uniprocessor (e.g., a basic scalar CPU); SIMD — a vector processor or GPU (e.g., Intel SSE/AVX vector unit, or a GPU executing the same instruction on many data elements); MISD — a systolic array or a pipeline of processors where multiple instructions operate on a single data stream (e.g., a systolic array for matrix multiplication); MIMD — a multicore processor or a cluster of workstations (e.g., Intel Core i7 multicore, or a distributed-memory cluster).

Unofficial marking guide:

- *1 mark: stating Flynn's taxonomy based on instruction and data streams*
- *1 mark: listing the four categories SISD, SIMD, MISD, MIMD*

© Hak Cipta Universiti Teknologi MARA

• 1 mark: correct example for SISD
• 1 mark: correct example for SIMD
• 1 mark: correct example for MISD and MIMD (one mark for both, or 0.5 each)
Model confidence: high.
Model note: Assumes the classification intended is Flynn's taxonomy, which is standard for this topic. Examples are illustrative; any valid example for each category should be accepted.

QUESTION A4 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique illustrated is direct addressing (also called absolute addressing). The instruction's address field A contains the effective address of the operand, so the operand is fetched directly from memory location A — the address field points straight at the operand, with no intermediate indirection.

(b) How it is used: The CPU decodes the instruction and takes the address field A as the effective address (EA = A). It then performs a single memory read at address A to fetch the operand into a register, after which the ALU executes the operation (e.g. ADD, LOAD) using that operand. Because the address is embedded in the instruction itself, no address calculation or extra memory access is needed to determine where the operand lives — the operand is obtained in one memory reference following instruction fetch.

(c) Advantage: It is simple and fast — only one memory access is required to fetch the operand (no extra indirection), and no address computation is needed. Disadvantage: The address field must be large enough to hold a full memory address, limiting the range of addressable memory (and the number of opcodes) for a fixed instruction length; it is also inflexible, since the address is fixed at assembly time and cannot easily support relocation or indexing.

Unofficial marking guide:

• 1 mark: Correctly names the technique as direct addressing (absolute addressing).
• 1 mark: States EA = A, i.e. the address field holds the operand's memory address.
• 1 mark: Explains the single memory access to fetch the operand and use it in the operation.
• 1 mark: Gives one valid advantage (e.g. simple/fast, one memory access, no address calculation).
• 1 mark: Gives one valid disadvantage (e.g. limited addressable range/short address field, inflexibility/no relocation or indexing).
Model confidence: medium.
Model note: Figure 2 is inferred from the extracted text (Instruction: Opcode | Address A; Memory: Operand) as direct addressing; if the figure actually shows the address field pointing to a memory cell that itself holds the operand's address, the answer would instead be indirect addressing. Assumed the standard interpretation of the shown layout.

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (10).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three types of interfaces to external devices:

1. Programmed I/O (polled I/O): The CPU directly controls the transfer and repeatedly checks the status of the device through a status register. The CPU issues a command, then polls the device until it is ready, and then transfers data one word at a time. This wastes CPU cycles because the processor is busy waiting, but it requires simple hardware and no interrupt mechanism.

2. Interrupt-driven I/O: The CPU issues a command and continues executing other instructions. When the device is ready, it raises an interrupt request; the CPU suspends the current program, saves context, and branches to an interrupt service routine (ISR) to transfer the data. This frees the CPU from polling, but each transfer still requires CPU involvement, so it is inefficient for large blocks.

3. Direct Memory Access (DMA): A dedicated DMA controller transfers blocks of data directly between the device and main memory without CPU intervention. The CPU initialises the DMA controller (address, count, direction) and is interrupted only when the whole block is complete. This is efficient for high-speed block devices such as disks and network cards.

(b) Two types of interrupt:

1. Maskable interrupt: An interrupt that can be disabled (masked) by the CPU, typically by clearing the

© Hak Cipta Universiti Teknologi MARA

interrupt-enable flag or by an interrupt mask register. It is used for ordinary I/O devices whose service can be delayed. When masked, the interrupt request is ignored until it is unmasked.

2. Non-maskable interrupt (NMI): An interrupt that cannot be disabled by the CPU. It is reserved for critical events such as hardware failure, power failure, or memory parity errors, and is always serviced immediately. It has higher priority than maskable interrupts.

(Alternatively, interrupts may be classified as hardware interrupts (external events) and software interrupts (internal exceptions/traps); either classification is acceptable if explained.)

Unofficial marking guide:

• 1 mark: naming Programmed I/O (polling) as an interface type
• 1 mark: describing programmed I/O operation and CPU busy-waiting
• 1 mark: naming interrupt-driven I/O as an interface type
• 1 mark: describing interrupt-driven I/O operation and ISR/context save
• 1 mark: naming DMA as an interface type
• 1 mark: describing DMA operation with DMA controller and block transfer
• 1 mark: naming a maskable interrupt
• 1 mark: explaining maskable interrupt (can be disabled/masked)
• 1 mark: naming a non-maskable interrupt (NMI)
• 1 mark: explaining NMI (cannot be disabled, critical events)

Model confidence: high.

Model note: The question does not specify a particular classification for interrupts; the answer uses maskable/non-maskable, which is standard. Hardware/software interrupt classification is also acceptable if explained. No figure or table is required.

QUESTION A6 (10 marks)

Topic: Data representation and number systems

a) Answer: -6.7510 = 0xC0D80000 (sign=1, exponent=10000001, fraction=10110000000000000000)

• 1 mark for sign, exponent and mantissa.
• 1 mark for the biased exponent.
• 1 mark for the 23-bit fraction.
• 1 mark for the 32-bit binary grouping.
• 1 mark for the hexadecimal result.

b) Answer: 0x42C90000 = 100.510

• 1 mark for the sign bit.
• 1 mark for the exponent field.
• 1 mark for the fraction field.
• 1 mark for applying the bias and significand.
• 1 mark for the final decimal value.

QUESTION A7 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (21).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) An I/O module is installed whenever a computer system needs to connect the CPU/memory to a peripheral device that cannot be attached directly to the system bus. It is installed at system configuration/build time (as part of the motherboard chipset or as an add-in/expansion card in a slot) and, more generally, whenever a new external device must be interfaced to the system — i.e. it is the hardware that sits between the system bus and the peripheral to perform control, buffering, and data transfer (and often error detection and format conversion). [2 marks]

(b) If the I/O module is not installed, the peripheral cannot be interfaced to the system bus, so the CPU cannot communicate with or control that device. The device's data cannot be transferred to/from memory, no buffering or format/speed matching is available, and the peripheral is effectively unusable (the system may also fail to detect the device or report an error). [2 marks]

(c) One category of I/O module is a 'controller/interface module' classified by its data-transfer role, e.g. a block-oriented (block device) I/O module. Explanation: a block-oriented I/O module transfers data in fixed-size blocks (e.g. disk, tape, USB mass storage); it typically supports direct memory access (DMA), so it can move a whole block between the device and main memory without the CPU handling every word, issuing an interrupt only when the transfer is complete. (Other acceptable categories: character-

© Hak Cipta Universiti Teknologi MARA

oriented/stream module for byte-at-a-time devices such as keyboards and printers; or a programmable/non-programmable, or dedicated/shared, I/O module.) [2 marks]

(d) 'Hot connection' (hot plugging/hot swapping) means a device or I/O module can be connected to or removed from a running system without powering the system down, and the system detects and configures it automatically. [1 mark]

(e) Three common interfaces to external devices: (1) USB (Universal Serial Bus); (2) SATA (Serial ATA) / eSATA; (3) PCI Express (PCIe) — also acceptable: FireWire (IEEE 1394), Thunderbolt, HDMI/DisplayPort, Ethernet, Bluetooth, SCSI/SAS. [3 marks]

Unofficial marking guide:

• 2 marks: (a) states I/O module is installed when a peripheral must be interfaced to the system bus / at system build or expansion time, with the role of interfacing CPU/memory to the device (1 mark for timing/context, 1 mark for purpose).

• 2 marks: (b) states the peripheral cannot be connected/controlled and data cannot be transferred (1 mark), plus consequence such as no buffering/format matching or device unusable/undetected (1 mark).

• 2 marks: (c) names one valid category of I/O module (1 mark) and explains its characteristic behaviour, e.g. block-oriented with DMA (1 mark).

• 1 mark: (d) correct definition of hot connection — connecting/removing a device while the system is powered/running, with automatic detection.

• 3 marks: (e) one mark for each correctly named common external-device interface (three required).

Model confidence: high.

Model note: OCR 'I/O' read as 'l/O'. Part (c) is open-ended; any one valid category (block-oriented, character-oriented, programmable, etc.) with a correct explanation is acceptable. No figure/table required.

QUESTION A8 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (7).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A computer bus is a shared communication pathway that connects the CPU, memory and I/O devices. The main bus types and their functions are: (i) Data bus – carries the actual data (instructions and operands) between the processor, memory and I/O; it is bidirectional and its width determines how many bits can be transferred at once. (ii) Address bus – carries the address of the memory location or I/O port being accessed; it is unidirectional (from CPU/master to memory or I/O) and its width determines the maximum addressable memory space. (iii) Control bus – carries control and timing signals such as read/write, memory request, I/O request, interrupt request/acknowledge, clock and bus grant, to coordinate and synchronise transfers on the data and address buses. (iv) Power bus – distributes power (and ground) to the system components. (Some texts also distinguish a system bus, which combines data, address and control lines, from an expansion/I/O bus such as PCI or USB used to attach peripheral devices.)

(b) Two benefits of synchronous bus timing: (i) Simplicity and low cost – all transfers are referenced to a common clock, so no complex handshaking or asynchronous control logic is needed; the bus protocol is simple and easy to implement. (ii) High speed/fast transfer – because events occur at fixed clock edges, transfers can be pipelined and run at a predictable, high rate, and the timing is easy to analyse and design for. (A further acceptable benefit is that synchronous timing is deterministic and easier to test/debug.)

Unofficial marking guide:

• 1 mark: correct function of data bus (carries data, bidirectional, width = bits per transfer)

• 1 mark: correct function of address bus (carries address, unidirectional, width = addressable space)

• 1 mark: correct function of control bus (carries read/write, timing, interrupt, clock signals)

• 1 mark: correct function of power bus (distributes power/ground)

• 1 mark: correct function of system bus (combines data/address/control) or expansion/I/O bus (connects peripherals)

• 1 mark: any one additional correct bus type or accurate supporting detail (e.g. bus width effect, master/slave roles)

• 2 marks: first benefit of synchronous timing (simplicity/low cost, no handshaking) with explanation

• 2 marks: second benefit of synchronous timing (high speed/predictable timing/pipelining) with explanation

Model confidence: high.

Model note: The question text does not list the specific bus types expected; the answer assumes the standard set (data, address, control, power, plus system/expansion bus). If the lecturer's list differs, map the same functional descriptions to the named buses. No figure or table is required.

© Hak Cipta Universiti Teknologi MARA

QUESTION A9 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (15).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Main function of cache memory [2 marks]
Cache memory is a small, fast, high-cost memory placed between the CPU and main memory. Its main function is to hold the most frequently and recently used instructions and data so that the CPU can access them with much lower latency than fetching from main memory. It exploits the principle of locality of reference (temporal and spatial locality): by keeping a copy of active blocks of main memory close to the processor, cache reduces the average memory access time and the gap between fast CPU speed and slower main memory, thereby improving overall system performance.

(b) Synchronizing data in cache and main memory [4 marks]
Because cache holds copies of main-memory blocks, writes must be kept consistent using a write policy:
- Write-through: every write to cache is also written immediately to main memory, so main memory is always up to date; simple but generates more memory traffic (often reduced with write buffers).
- Write-back (copy-back): writes update only the cache line and set a dirty bit; the block is written back to main memory only when it is replaced/evicted. This reduces memory traffic but requires a dirty bit per line and a write-back on eviction.
Consistency is also maintained by cache coherence protocols (e.g. snooping/MESI) in multiprocessor systems so that all caches see a consistent value, and by using valid/dirty bits so stale or modified lines are correctly handled on replacement.

(c) SRAM vs DRAM [4 marks]
- Type of storage: SRAM stores each bit in a bistable flip-flop/latch (6-transistor cell); DRAM stores each bit as charge on a capacitor (1 transistor + capacitor).
- Availability of charge leak: SRAM has no charge-leak problem (state held as long as power is on); DRAM cells leak charge and must be periodically refreshed.
- Price: SRAM is expensive per bit (larger cell, lower density); DRAM is much cheaper per bit (small cell, high density).
- Example of location of memory usage: SRAM is used for cache memory and registers; DRAM is used for main memory (RAM modules).

Unofficial marking guide:

• 1 mark: cache is small/fast memory between CPU and main memory holding frequently/recently used data and instructions
• 1 mark: explains locality of reference and/or reduction of average memory access time improving performance
• 1 mark: write-through policy explained (write to cache and main memory simultaneously)
• 1 mark: write-back policy explained (dirty bit, write back on eviction)
• 1 mark: role of valid/dirty bits or cache coherence in keeping copies consistent
• 1 mark: SRAM type of storage (flip-flop/latch) and DRAM type (capacitor)
• 1 mark: SRAM no charge leak vs DRAM requires refresh due to charge leak
• 1 mark: SRAM expensive per bit vs DRAM cheap per bit
• 1 mark: SRAM used in cache/registers vs DRAM used in main memory

Model confidence: high.
Model note: Question text has OCR artefacts ('ii,' and unnumbered list items in part c); treated as four characteristics: type of storage, charge leak, price, example location. Marking guide is unofficial and allocates 2+4+4 = 10 marks.

QUESTION A10 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (17).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Word size: The word size is 32 bits. Justification: the word size of a machine is conventionally the width of its data bus (the number of bits that can be transferred to/from memory or an I/O device in one bus transaction). Since the data bus is 32 bits, the natural word size is 32 bits (4 bytes). The 8-bit address bus determines the number of addressable locations (2^8 = 256 locations), not the word width, and the 4-bit control bus carries control signals, not data.

(b) Register size: The register size is 32 bits. Justification: general-purpose registers in a CPU are normally the same width as the machine word so that a single register can hold one data-bus word and

© Hak Cipta Universiti Teknologi MARA

can be loaded/stored in one memory access. With a 32-bit data bus, registers are 32 bits wide; the 8-bit address bus only affects the size of the address register/program counter (8 bits) and the addressable memory space, and the 4-bit control bus only affects control-signal width.

(c) If the system interconnection (the shared bus/system bus) is blocked: The machine cycle consists of fetch, decode, execute (and possibly memory/IO and write-back) phases. During the fetch phase the CPU must place the address on the address bus and read the instruction from memory over the data bus; if the interconnection is blocked, the instruction cannot be fetched, so the fetch phase cannot complete and the CPU stalls (inserts wait states) or times out with a bus error. Without a fetched instruction there is no decode or execute, so the machine cycle cannot progress. Similarly, operand reads/writes and I/O transfers in the execute/memory phases cannot occur, so any program requiring memory access halts. Effectively the CPU is isolated from memory and I/O, and the system cannot run programs; only operations using internal registers could continue, but even the next instruction fetch would fail.

(d) Role of CPU interconnection: CPU interconnection (the internal bus/CPU bus or the system bus linking CPU, memory and I/O) provides the communication path for address, data and control signals. It allows the CPU to send addresses to memory/I/O, transfer instructions and data to/from the CPU, and carry control signals (read/write, clock, interrupt, reset) that synchronize and coordinate the transfer. It thus enables the fetch-execute machine cycle and the movement of data between the CPU and the rest of the computer system.

Unofficial marking guide:

• 1 mark: word size stated as 32 bits
• 1 mark: justification that word size equals data bus width (32-bit data bus), not address/control bus
• 1 mark: register size stated as 32 bits
• 1 mark: justification that registers match the machine word/data bus width; address register/PC is 8 bits due to address bus
• 1 mark: identifies machine cycle phases (fetch, decode, execute, memory/IO)
• 1 mark: explains fetch cannot complete because address/instruction cannot be transferred over blocked interconnection
• 1 mark: explains CPU stalls/waits or bus error, and decode/execute cannot proceed
• 1 mark: explains operand/memory/IO transfers fail so program execution halts
• 1 mark: role of CPU interconnection as communication path for address, data and control signals
• 1 mark: explains it enables fetch-execute cycle and CPU-memory-I/O data transfer/synchronization

Model confidence: high.
Model note: Assumes 'system interconnection' means the shared system bus and 'CPU interconnection' means the CPU/system bus. Word size is taken as data bus width, which is the standard convention for this type of question.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (14).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation: Figure 4 is read as z = A + B² − C(D − C/B), i.e. B2 is taken as B squared (B^2). The expression is evaluated in the order: t1 = C/B; t2 = D − t1; t3 = C × t2; t4 = B × B; t5 = A + t4; z = t5 − t3. Registers A, B, C, D are read-only; x and y are scratch; the result is left in z.

Zero-address program (stack machine). Each operand is pushed, the operation consumes the top two stack entries and pushes the result; POP stores the top of stack into the named register.

```
; z = A + B^2 - C*(D - C/B)
PUSH C      ; stack: C
PUSH B      ; stack: C, B
DIV         ; stack: C/B           (t1)
PUSH D      ; stack: t1, D
SUB         ; stack: D - C/B       (t2)
PUSH C      ; stack: t2, C
MUL         ; stack: C*(D - C/B)   (t3)
PUSH B      ; stack: t3, B
PUSH B      ; stack: t3, B, B
MUL         ; stack: t3, B^2       (t4)
PUSH A      ; stack: t3, t4, A
ADD         ; stack: t3, A + B^2   (t5)
SUB         ; stack: (A + B^2) - t3
POP z       ; z = A + B^2 - C*(D - C/B)
```

One-address program (accumulator machine). The accumulator holds one operand; LOADx copies x into the accumulator, OPx applies the operation between the accumulator and x, and STORx writes the accumulator to x.

```
; z = A + B^2 - C*(D - C/B)
LOAD C      ; ACC = C
DIV B       ; ACC = C/B            (t1)
STOR x      ; x = C/B
LOAD D      ; ACC = D
SUB x       ; ACC = D - C/B        (t2)
STOR x      ; x = D - C/B
LOAD C      ; ACC = C
MUL x       ; ACC = C*(D - C/B)    (t3)
STOR x      ; x = t3
LOAD B      ; ACC = B
MUL B       ; ACC = B^2            (t4)
ADD A       ; ACC = A + B^2        (t5)
SUB x       ; ACC = (A + B^2) - t3
STOR z      ; z = final answer
```

Two-address program (register/memory-register machine). MOVE(x,y) copies y into x; OP(x,y) performs x = x OP y, so the destination register is overwritten and must be chosen carefully because A, B, C, D are read-only.

```
; z = A + B^2 - C*(D - C/B)
MOVE x, C   ; x = C
DIV  x, B   ; x = C/B              (t1)
MOVE y, D   ; y = D
SUB  y, x   ; y = D - C/B          (t2)
MOVE x, C   ; x = C
MUL  x, y   ; x = C*(D - C/B)      (t3)
MOVE y, B   ; y = B
MUL  y, B   ; y = B^2              (t4)
```

© Hak Cipta Universiti Teknologi MARA

```
MOVE z, A   ; z = A
ADD  z, y   ; z = A + B^2        (t5)
SUB  z, x   ; z = (A + B^2) - t3  = final answer
```

In all three versions the final result is held in z, and A, B, C, D are never used as destinations, satisfying the constraints.

Unofficial marking guide:

• 4 marks: zero-address program — correct PUSH/POP/ADD/SUB/MUL/DIV sequence, correct operator order for C/B, D−C/B, C×(…), B×B, A+B², final subtraction, and POP z

• 4 marks: one-address program — correct LOAD/STOR/OPx sequence with accumulator handling and correct use of x as temporary

• 4 marks: two-address program — correct MOVE/OP(x,y) sequence with correct destination choice and no modification of A, B, C, D

• 3 marks: correct evaluation order/parenthesisation of z = A + B² − C(D − C/B) in all three programs (division before subtraction, multiplication before final subtraction)

• 3 marks: correct assembly notation, comments and final result stored in z in each program

• 2 marks: no read-only register (A, B, C, D) is overwritten and only x, y, z are used as temporaries

Model confidence: medium.

Model note: Figure 4 is only available as extracted text; 'B 2' is assumed to mean B squared (B^2). If the original figure shows B×2 instead, replace each B×B / MUL B step with a multiply by constant 2. Table 3 OCR is imperfect ('T\vr.-address' = two-address); instruction mnemonics are interpreted conventionally: zero-address PUSH/POP/ADD/SUB/MUL/DIV, one-address LOADx/STORx/ADDx/SUBx/MULx/DIVx, two-address MOVE(x,y)/ADD(x,y)/SUB(x,y)/MUL(x,y)/DIV(x,y). This is an unofficial model answer, not a marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA